

## Lab 5 – FIRST and FOLLOW Computation

### Aim

To compute the FIRST and FOLLOW sets for the given grammar.

### Algorithm

#### FIRST

1. Start the program.
2. Read the grammar productions.
3. For a non-terminal **A**, examine each production.
4. If the first symbol is a terminal, include it in **FIRST(A)**.
5. If the first symbol is a non-terminal, include **FIRST** of that non-terminal.
6. If  $\epsilon$  occurs, continue with the next symbol.
7. Repeat until no new symbols are added.

#### FOLLOW

1. Add \$ to the **FOLLOW** of the start symbol.
2. For every production  $A \rightarrow \alpha B \beta$ :
  1. Add **FIRST( $\beta$ )** (excluding  $\epsilon$ ) to **FOLLOW(B)**.
3. If  $\beta$  is empty or can derive  $\epsilon$ :
  1. Add **FOLLOW(A)** to **FOLLOW(B)**.
4. Repeat until no changes occur.
5. Stop.

### C Program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 10

char production[MAX][20];
char first[MAX], follow[MAX];
int n;
int firstIndex, followIndex;

void addToSet(char *set, int *index, char c) {
    for (int i = 0; i < *index; i++) {
        if (set[i] == c)
            return;
    }
    set[*index] = c;
    (*index)++;
}
```

```

    }
    set[( *index)++] = c;
}

```

```

void FIRST(char c);
void FOLLOW(char c);

```

```

void FIRST(char c) {
    if (!isupper(c)) {
        addToSet(first, &firstIndex, c);
        return;
    }

    for (int i = 0; i < n; i++) {
        if (production[i][0] == c) {
            if (production[i][2] == '#') {
                addToSet(first, &firstIndex, '#');
            } else {
                FIRST(production[i][2]);
            }
        }
    }
}

```

```

void FOLLOW(char c) {
    if (production[0][0] == c)
        addToSet(follow, &followIndex, '$');

    for (int i = 0; i < n; i++) {
        int len = strlen(production[i]);

        for (int j = 2; j < len; j++) {
            if (production[i][j] == c) {
                if (production[i][j + 1] != '\0') {
                    if (!isupper(production[i][j + 1])) {
                        addToSet(follow, &followIndex,
                                production[i][j + 1]);
                    } else {
                        char temp[MAX];
                        int tempIndex = 0;

                        memset(first, 0, sizeof(first));
                        firstIndex = 0;
                        FIRST(production[i][j + 1]);

                        for (int k = 0; k < firstIndex; k++) {
                            if (first[k] != '#')
                                addToSet(follow, &followIndex, first[k]);
                        }
                    }
                }
            }
        }
    }
}

```

```

        } else if (production[i][0] != c) {
            FOLLOW(production[i][0]);
        }
    }
}

int main() {
    char symbol;

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (Example: E=TR)\n");
    for (int i = 0; i < n; i++)
        scanf("%s", production[i]);

    printf("Enter symbol to find FIRST and FOLLOW: ");
    scanf(" %c", &symbol);

    firstIndex = 0;
    memset(first, 0, sizeof(first));
    FIRST(symbol);

    printf("\nFIRST(%c) = { ", symbol);
    for (int i = 0; i < firstIndex; i++)
        printf("%c ", first[i]);
    printf("}\n");

    followIndex = 0;
    memset(follow, 0, sizeof(follow));
    FOLLOW(symbol);

    printf("FOLLOW(%c) = { ", symbol);
    for (int i = 0; i < followIndex; i++)
        printf("%c ", follow[i]);
    printf("}\n");

    return 0;
}

```

## Sample Input

```

Enter number of productions: 3
E=TR
R=+TR
R=#
Enter symbol to find FIRST and FOLLOW: E

```

## **Sample Output**

$\text{FIRST}(E) = \{ T \}$   
 $\text{FOLLOW}(E) = \{ \$ \}$

## **Result**

Thus the FIRST and FOLLOW sets for the given grammar were computed successfully.